expert_selection.py — technical

`src/moe_pruning/pruning/expert_selection.py · 82 lines · moe_pruning.pruning.expert_selection`

Purpose

Score rows and per-layer selection.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class ExpertScore`

lines 13-30 · dataclass

One row of expert_scores.json/.csv.

Field	Type	Default
`layer_id`	str	
`stack`	str	
`expert_id`	int	
`importance`	float	
`importance_share`	float	
`rank`	int	
`token_count`	int	
`routing_frequency`	float	
`avg_gate`	float | None	
`avg_output_norm`	float | None	
`avg_token_contribution`	float | None	
`observed`	bool	
`layer_selected_for_pruning`	bool	
`selected_for_replacement`	bool	

`ExpertScore.to_dict`

```
def to_dict(self) -> dict
```

lines 29-30

Convert to a plain dictionary for JSON reports.

`score_and_select`

```
def score_and_select(stats: dict[str, LayerStatistics], layers: list[MoELayerInfo], prune_layers:
list[MoELayerInfo], experts_per_layer: int, allow_unobserved: bool) -> tuple[list[ExpertScore],
dict[str, list[int]]]
```

lines 37-82

Rank experts independently in every layer; select in `prune_layers` only.

1. For every discovered layer: rank by importance (`rank_experts`).
2. If the layer is in the prune set: `select_lowest(importance, observed, k, allow_unobserved)`.
3. Emit one `ExpertScore` per expert: averages divide sums by counts (None when count = 0); importance share = I_e / sum I; routing frequency = count / real tokens.